

Assignment 4

1. 对一个包含 12 个元素的有序表进行折半查找（假设向下取整，即 $\text{mid} = \lfloor (\text{low} + \text{high}) / 2 \rfloor$ ）。如果将该查找过程抽象为一棵折半查找判定树，则关于该判定树的形态，下列说法中正确的是（ ）
 - A. 该判定树必然是一棵完全二叉树
 - B. 该判定树必然是一棵平衡二叉树（AVL 树）
 - C. 该判定树的最深一层必定是满的
 - D. 该判定树中可能存在度为 1 的结点，且该结点的左子树为空
2. 在一个初始为空的平衡二叉树（AVL 树）中依次插入关键字序列 {10, 20, 30, 40, 50, 60}。完成所有插入操作并经过必要的平衡调整后，该 AVL 树的根结点是（ ）
 - A. 20
 - B. 30
 - C. 40
 - D. 50
3. 在哈希表（散列表）中处理冲突时，采用线性探测再散列法往往容易产生“聚集（堆积）”现象，从而严重降低查找效率。导致这种聚集现象的最根本的直接原因是（ ）
 - A. 哈希函数的设计不均匀，导致映射地址集中
 - B. 散列表的装填因子（Load Factor）设置得过大
 - C. 探查序列是物理连续的，使得不同哈希映射值的元素争夺同一片连续的地址空间
 - D. 数据集中存在大量同义词（即具有相同哈希值的不同关键字）
4. 已知一棵高度为 4 的 5 阶 B 树（规定根结点所在的层数为第 1 层）。为了保持 B 树的性质，这棵 B 树中最多可能包含的关键码（关键字）总数是（ ）
 - A. 255
 - B. 312
 - C. 624
 - D. 780
5. 在设计散列表（Hash Table）时，关于平均查找长度（ASL）的理论分析，下列断言中最准确的是（ ）
 - A. 只要散列表的长度 m 足够大，ASL 就可以无限逼近于 1
 - B. 散列表的 ASL 仅取决于哈希函数，与冲突解决策略无关
 - C. 散列表的 ASL 是装填因子 α 的函数，在哈希函数和冲突处理策略确定的前提下，ASL 并不直接依赖于表长 m 或元素总数 n 的绝对大小
 - D. 采用链地址法（拉链法）处理冲突时，其 ASL 永远小于采用开放定址法的散列表
6. 对一个包含 n 个元素的线性表采用分块查找（索引顺序查找）。设表被均匀分成了 s 个块，每个块内有 b 个元素（即 $n = s \times b$ ）。若在索引表和块内部均采用顺序查找策略，为了使全表的平均查找长度（ASL）达到理论最小值，理想情况下每个块的大小 b 应被设计为（ ）
 - A. $\log_2 n$
 - B. $n/2$
 - C. $n^{1/2}$
 - D. n
7. 在现代关系型数据库（如 MySQL 的 InnoDB 引擎）和操作系统文件系统中，底层索引结构几乎无一例外地选择了 B+ 树而非 B 树。这主要是因为 B+ 树具备以下哪个绝对优势？（ ）

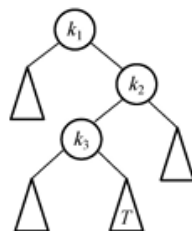
- A. B+ 树的插入和删除操作永远不会引起结点的分裂或合并
- B. B+ 树的非叶子结点不存储实际数据，仅作为索引，因此同等磁盘页大小下能容纳更多索引项，树变得更矮胖，大幅降低了磁盘 I/O 次数
- C. B+ 树的查找速度在任何情况下都快于 B 树
- D. B+ 树的所有关键码都是互不相同的，天然去重
8. 在一棵二叉排序树 (BST) 中删除一个同时拥有左子树和右子树的结点 P 时，为了尽量减小树的形态变动并维持排序特性，标准操作是找到一个替身结点 S 的值覆盖 P 的值，然后将 S 结点物理删除。这个替身结点 S 必定是 ()
- A. P 的左子树中值最大的结点，或右子树中值最小的结点
- B. P 的直接左孩子，或直接右孩子
- C. 整个二叉排序树的根结点
- D. P 的右子树中深度最深的叶子结点
9. 设哈希函数为 $H(\text{key}) = \text{key} \pmod{7}$ ，哈希表的地址空间为 0 ~ 6。将关键字序列 {15, 22, 29, 36} 依次插入到该空表中，采用线性探测法解决冲突。在成功插入这四个元素后，元素 36 所在的数组下标位置是 ()
- A. 1
- B. 2
- C. 3
- D. 4
10. 如果将一个给定的随机关键字序列依次插入到一棵空的二叉排序树 (BST) 中，最终构建出的树形结构的高低 (深度) 严重依赖于输入序列的特征。要使生成的二叉排序树达到最坏的时间复杂度退化 (即深度最深，变成单链表)，输入序列必须具备的特征是 ()
- A. 序列中的关键字完全随机，毫无规律
- B. 序列必须严格升序排列
- C. 序列必须严格降序排列
- D. 序列是严格有序的 (无论是严格升序还是严格降序)
11. 在构建用于大语言模型 (LLM) 推理的分布式 KV 缓存系统时，工程师为某个高频路由表设计了一个驻留在内存中的散列表 (Hash Table)。为了追求极致的连续内存访问速度 (Cache Friendly)，放弃了拉链法，改用开放定址法 (线性探测再散列) 来解决哈希冲突。在系统运行高峰期，某个过期的 Token 路由节点 A 需要被删除。假设路由表当前非常拥挤，且有多个其他节点的哈希值与 A 冲突，并顺延存放在 A 后面的位置
- 关于节点 A 的删除操作，以下在 408 理论与工程逻辑上绝对正确的是 ()
- A. 必须直接将 A 所在的数组槽位清空 (置为 NULL)，以便后续新插入的元素可以立刻复用该空间，最大化内存利用率
- B. 绝对不能直接清空 A 的槽位，必须使用“墓碑机制 (Tombstone)”将其标记为“已逻辑删除”，否则会导致部分后续冲突元素的查找链条断裂
- C. 删除 A 后，必须立刻触发一次全表的 Rehash (重散列)，将 A 后面的所有冲突元素重新计算位置并向前移动，以保证查找效率
- D. 在线性探测法中，查找失败的平均查找长度 (ASL) 仅取决于哈希函数的设计，与是否删除元素、如何删除元素无关
12. 在处理 2024 年纽约黄出租车 (NYC Yellow Taxi) 的海量 OD (Origin-Destination) 时空网络数据时，需要将数亿条出行记录持久化到磁盘数据库中
- 分析师最常用的查询模式是范围检索，例如：“提取特定时间段 (如 08:00 至 09:00) 内的所有订单数据”。为了优化这类涉及海量磁盘 I/O 的查询，底层存储引擎使用了 B+ 树而不是 B 树作为主键索引
- 关于 B+ 树在此类数据分析场景下的核心优势，以下说法最不准确的是 ()

- A. B+ 树的非叶子节点不存储实际的出租车订单数据（仅存索引关键字），因此一个磁盘页可以装载更多的索引项，树的高度更矮，磁盘 I/O 次数更少
- B. 在查询 08:00 到 09:00 的数据时，B+ 树只需通过树根进行一次 $O(\log_m N)$ 的定位找到 08:00 的叶子节点，随后沿着叶子节点的链表顺序遍历即可，而 B 树必须反复进行中序遍历
- C. B+ 树的所有叶子节点包含了全部关键字信息，且叶子节点本身按关键字大小链接，构成了一个有序的单链表（或双向链表）
- D. B 树由于在非叶子节点也能直接命中并返回出租车记录，因此在进行“08:00 至 09:00”这种大范围数据提取时，其平均时间复杂度通常优于 B+ 树
13. 在基于华为昇腾（Ascend C）的底层计算资源调度中，操作系统内核需要维护一个完全公平调度器（CFS），用于管理成千上万个高并发创建与销毁的轻量级算子任务
- 系统工程师决定使用红黑树（Red-Black Tree）而不是 AVL 树（平衡二叉树）来作为这些任务控制块的底层查找目录
- 促使工程师做出这一底层数据结构选型的最核心数学/理论原因是（ ）
- A. 红黑树的查找时间复杂度是严格的 $O(1)$ ，而 AVL 树是 $O(\log N)$
- B. 红黑树在进行插入和删除操作时，虽然也需要调整，但其引发的结构旋转次数具有极其优秀的常数级上界（删除最多 3 次旋转），非常适合“写多读多”的高频动态环境
- C. AVL 树要求任何节点的左右子树高度差绝对值不超过 1，这导致其节点中必须额外存储大量的平衡因子指针，极大地浪费了 NPU 的 L1 Cache
- D. 红黑树的根节点到叶子节点的所有路径长度绝对相等，不存在 AVL 树那种路径长短不一的情况，保证了算子调度的绝对公平
14. 在大语言模型推理系统的 KV Cache 显存池管理中，为了加速空闲显存块的匹配，系统将当前可用的 12 个大小不等的连续显存块的物理地址，按容量从小到大排序，存放在一个长度为 12 的数组中
- 现在使用折半查找（Binary Search）算法来搜索一个容量精准匹配的内存块。如果将该折半查找的过程画作一棵判定树（Decision Tree）（向下取整规则： $\text{mid} = \lfloor (\text{low} + \text{high})/2 \rfloor$ ）。关于这棵折半查找判定树的数学属性，以下推算绝对正确的是（ ）
- A. 该判定树是一棵满二叉树，所有叶子节点都处于同一深度
- B. 查找失败时，最多需要进行 5 次关键字的比较
- C. 该判定树中包含了 12 个内部节点（代表查找成功），以及 13 个外部节点（方形节点，代表查找失败）
- D. 在该判定树中，任意一个节点的左子树的节点总数，必定严格等于右子树的节点总数
15. 在 LLM 的推测解码（Speculative Decoding）中，小模型会快速生成一段 Token 序列（模式串），大模型随后并行验证这段序列（主串）。如果发生 Token 不匹配，系统需要快速回退并找到下一个最可能的匹配起点
- 这在数学原理上与 KMP 字符串模式匹配算法中的 `next` 数组（失效函数）极为相似。假设小模型生成的 Token 模式串为 `S = "ababaaababaa"`
- 基于 408 统考的标准求法（数组下标从 1 开始，`next[1]=0`，`next[2]=1`），请计算该模式串的 `next` 数组。当匹配到最后一个字符 'a' 发生失败时，指针应该回退到模式串的第（ ）个位置？
- A. 4
- B. 5
- C. 6
- D. 7
16. 25. 【2024 统考真题】下列数据结构中，不适合直接使用折半查找的是（ ）。
- I. 有序链表 II. 无序数组 III. 有序静态链表 IV. 无序静态链表
- A. 仅 I、III B. 仅 II、IV C. 仅 II、III、IV D. I、II、III、IV

26. 【2025 统考真题】已知查找表中有 400 个元素，查找每个元素的概率相同。采用分块查找法进行查找，且均匀分块。若采用顺序查找法确定元素所在的块，且块内也采用顺序查找法，为使查找效率最高，则每块包含的元素个数应为（ ）。
17. A. 8 B. 10 C. 20 D. 25

34. 【2024 统考真题】一棵二叉搜索树如右图所示， k_1 、 k_2 、 k_3 分别是对应结点中保存的关键字。子树 T 的任一结点中保存的关键字 x 满足的是（ ）。

18. A. $x < k_1$ B. $x > k_2$
C. $k_1 < x < k_3$ D. $k_3 < x < k_2$



二、综合应用题

25. 【2025 统考真题】给定 7 个不同的关键字，能构造的不同 4 阶 B 树的个数最多是（ ）。
19. A. 7 B. 8 C. 9 D. 10

24. 【2025 统考真题】下列关于散列方法处理冲突的叙述中，正确的是（ ）。
20. A. 只要散列表不满，线性探查再散列一定能找到一个空闲位置
B. 只要散列表不满，二次探查再散列一定能找到一个空闲位置
C. 线性探查再散列处理的冲突，一定是发生在同义词之间的冲突
D. 二次探查再散列处理的冲突，一定是发生在非同义词之间的冲突

12. 【2024 统考真题】KMP 算法使用修正后的 next 数组进行模式匹配，模式串为 $S = 'aabaab'$ ，当主串的某个字符与 S 的某个字符失配时， S 向右滑动的最长距离是（ ）。
21. A. 5 B. 4 C. 3 D. 2

2010年真题

应用题

41. (10 分) 将关键字序列 (7, 8, 30, 11, 18, 9, 14) 散列存储到散列表中。散列表的存储空间是一个下标从 0 开始的一维数组，散列函数为 $H(\text{key}) = (\text{key} \times 3) \bmod 7$ ，处理冲突采用线性探测再散列法，要求装填 (载) 因子为 0.7。
22. 1) 请画出所构造的散列表。
2) 分别计算等概率情况下查找成功和查找不成功的平均查找长度。

2013年真题

应用题

42. (10 分) 设包含 4 个数据元素的集合 $S = \{ "do", "for", "repeat", "while" \}$ ，各元素的查找概率依次为 $p_1 = 0.35$, $p_2 = 0.15$, $p_3 = 0.15$, $p_4 = 0.35$ 。将 S 保存在一个长度为 4 的顺序表
23. 中，采用折半查找法，查找成功时的平均查找长度为 2.2。请回答：

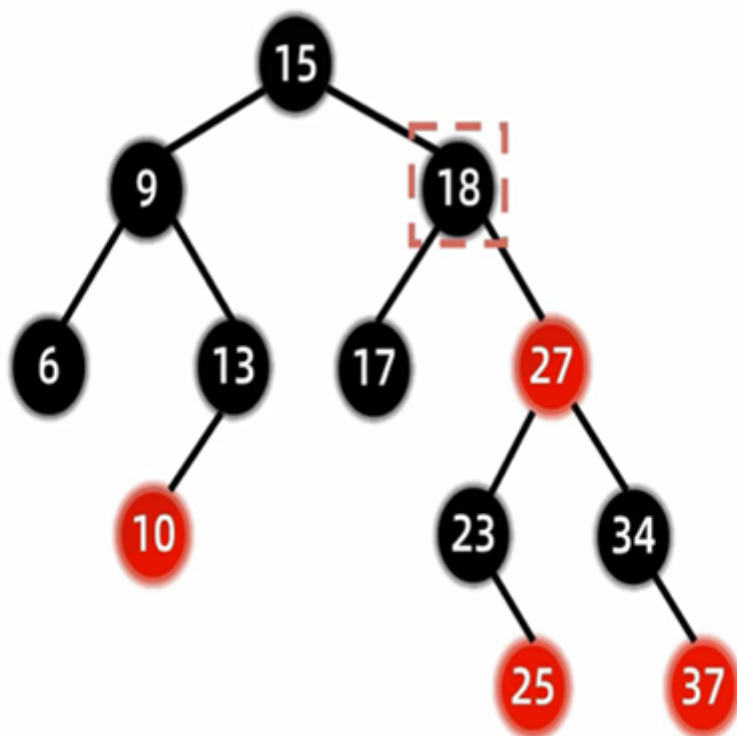
- (1) 若采用顺序存储结构保存 S ，且要求平均查找长度更短，则元素应如何排列？应使用何种查找方法？查找成功时的平均查找长度是多少？
(2) 若采用链式存储结构保存 S ，且要求平均查找长度更短，则元素应如何排列？应使用何种查找方法？查找成功时的平均查找长度是多少？

07. 【2024 统考真题】将关键字序列 20, 3, 11, 18, 9, 14, 7 依次存储到初始为空、长度为 11 的散列表 HT 中，散列函数 $H(\text{key}) = (\text{key} \times 3) \% 11$ 。 $H(\text{key})$ 计算出的初始散列地址为 H_0 ，发生冲突时探查地址序列是 $H_1, H_2, H_3, \dots$ ，其中， $H_k = (H_0 + k^2) \% 11$, $k = 1, 2, 3, \dots$ 。
24. 请回答下列问题：
- 1) 画出所构造的 HT，并计算 HT 的装填因子。
2) 给出在 HT 中查找关键字 14 的关键字比较序列。
3) 在 HT 中查找关键字 8，确认查找失败时的散列地址是多少？

25. 按顺序删除 18、25、15、6 结点

红黑树删除示例

18 25 15 6 13 37 27 17 34 9 10 23



B 树 插入（先插入后分裂）

26. 关键字序列为：(1, 2, 6, 7, 11, 4, 8, 13, 10, 5, 17, 9, 16, 20, 3, 12, 14, 18, 19, 15), 创建一棵 5 阶 B 树。

$m = 5$

27. B 树 删除示例

对于前例生成的 B 树，给出删除 8, 16, 15, 4 等 4 个关键字的过程

28. 已知一个长度为 n 的升序整数数组 A ，其中可能包含重复元素。请设计一个时间复杂度尽可能高效的算法，统计给定元素 k 在数组 A 中出现的次数

请完成下列任务：

1. 给出算法的基本设计思想
 2. 采用 C 或 C++ 语言描述算法，关键之处给出注释
 3. 分析你所设计算法的时间复杂度和空间复杂度
29. 已知一棵二叉搜索树（二叉排序树）采用二叉链表存储结构，每个结点存放一个互不相同的整数关键字。请设计一个算法，输出该二叉搜索树中所有关键字不小于 x 的结点的数据值，并且要求输出序列按从大到小的顺序排列

请完成下列任务：

- 1.给出算法的基本设计思想
- 2.写出该二叉搜索树的链式存储结构定义（C/C++ 语言）
- 3.根据设计思想，采用 C 或 C++ 语言描述算法
- 4.分析你所设计算法的时间复杂度

30. 顺序查找

折半查找

分块查找

二叉树搜索树

定义

查找

插入

平衡二叉树

定义

LL型调整：右单旋转

RR型调整：左单旋转

KMP算法

求 Next 数组

求 Nextval 数组

KMP 模式匹配